Language Models for Code

Presented by: Matt Juntima (vqj9sq), Shao, Lize (zgr3et) and Grayson Xie (chh8pj)

Overview

- Language Models are being heavily integrated in the SDLC
- Tasks can vary from automation, documentation filling, code filling, debugging, or issue solving.

Papers:

1. InCoder: A Generative Model for Code Infilling and Synthesis
2. Teaching Large Language Models to Self-Debug
3. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering

InCoder: A Generative Model for Code Infilling and Synthesis

Daniel Fried[*♡†◇] **Armen Aghajanyan**[*♡] **Jessy Lin**[♣]
Sida Wang[♡] **Eric Wallace**[♣] **Freda Shi**[△] **Ruiqi Zhong**[♣]
Wen-tau Yih[♡] **Luke Zettlemoyer**[♡†] **Mike Lewis**[♡]

Facebook AI Research[♡] University of Washington[†]
UC Berkeley[♣] TTI-Chicago[△] Carnegie Mellon University[◇]

dfried@cs.cmu.edu, {armenag,mikelewis}@fb.com

Presented by: Matt Juntima

Background

- Code written in the real-world is rarely written in a single left-to-right pass and is instead repeatedly edited and refined
- Less directly applicable to many ubiquitous code editing tasks, such as fixing bugs, adding comments, or re-naming variables
- Most Language learning models generate code from left-to-right.
- Solution: **InCoder** - LLM that generates code with context from left AND right parts of the code (Casual Masking)

Training Process

- 6.7B Transformer model, trained on 248 V100 GPUs for 24 days
- Collection of public code with permissive, non-copyleft, opensource licenses from GitHub and GitLab and StackOverflow questions, answers, and comments.

- Evaluation Methods:
  - **Pass Rate:** the rate at which the completed function passes all of the function's input–output pairs
  - **Exact Match:** percentage of times that the completed lines exactly match the masked lines in the original implementation

Training: $\log P(\texttt{[Left; <Mask:0>; Right; <Mask:0>; Span; <EOM>]})$

Inference: $P(\cdot \mid \texttt{[Left; <Mask:0>; Right; <Mask:0>]})$

Infill Generation Example

Inference logic:

- “Eren” is likely to be a name
- “18” is age
- “Male” is the gender

```
class Person:

    def __init__(self,name,age,gender):
        self.name=name
        self.age=age
        self.gender=gender

p = Person('Eren', 18, "Male")
```

Training

Original Document

```
def count_words(filename: str) -> Dict[str, int]:
    """Count the number of occurrences of each word in the file."""
    with open(filename, 'r') as f:
        word_counts = {}
        for line in f:
            for word in line.split():
                if word in word_counts:
                    word_counts[word] += 1
                else:
                    word_counts[word] = 1
    return word_counts
```

Masked Document

```
def count_words(filename: str) -> Dict[str, int]:
    """Count the number of occurrences of each word in the file."""
    with open(filename, 'r') as f:
        <MASK:0> in word_counts:
                    word_counts[word] += 1
                else:
                    word_counts[word] = 1
    return word_counts
<MASK:0> word_counts = {}
        for line in f:
            for word in line.split():
                if word <EOM>
```

Zero-shot Inference

Type Inference

```
def count_words(filename: str) -> Dict[str, int]:
    """Count the number of occurrences of each word in the file."""
    with open(filename, 'r') as f:
        word_counts = {}
        for line in f:
            for word in line.split():
                if word in word_counts:
                    word_counts[word] += 1
                else:
                    word_counts[word] = 1
    return word_counts
```

Variable Name Prediction

```
def count_words(filename: str) -> Dict[str, int]:
    """Count the number of occurrences of each word in the file."""
    with open(filename, 'r') as f:
        word_count = {}
        for line in f:
            for word in line.split():
                if word in word_count:
                    word_count[word] += 1
                else:
                    word_count[word] = 1
    return word_count
```

Docstring Generation

```
def count_words(filename: str) -> Dict[str, int]:
    """
    Counts the number of occurrences of each word in the given file.

    :param filename: The name of the file to count.
    :return: A dictionary mapping words to the number of occurrences.
    """
    with open(filename, 'r') as f:
            word_counts = {}
            for line in f:
                for word in line.split():
                    if word in word_counts:
                        word_counts[word] += 1
                    else:
                        word_counts[word] = 1
        return word_counts
```

Multi-Region Infilling

```
from collections import Counter

def word_count(file_name):
    """Count the number of occurrences of each word in the file."""
    words = []
    with open(file_name) as file:
        for line in file:
            words.append(line.strip())
    return Counter(words)
```

Experimentation

- Zero-shot infilling for a diverse set of tasks: inserting lines of code, predicting function return types, generating docstrings, renaming variables, and inserting missing code tokens
- 3 different inference methods:
    1. Causal masking
    2. Standard left-to-right generation approach (left-to-right single)
    3. Left-to-right generation and reranking approach (left-to-right reranking)

Benchmark - HumanEval dataset

Infilling Results

Causal Mask infilling performs better in both single and multi-line infilling

Method	Pass Rate	Exact Match
L-R single	48.2	38.7
L-R reranking	54.9	44.1
CM infilling	69.0	56.3
PLBART	41.6	—
code-cushman-001	53.1	42.0
code-davinci-001	63.0	56.0

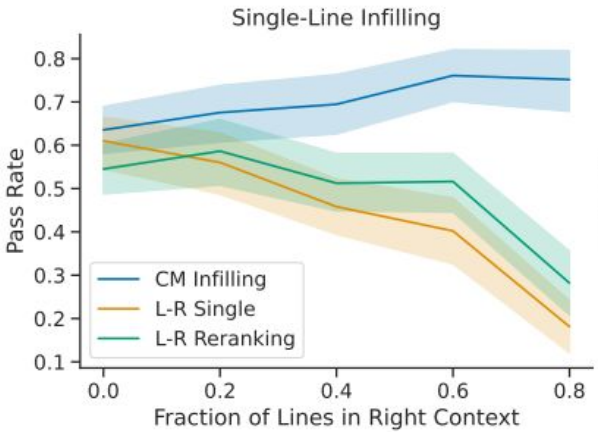
(a) Single-line infilling.

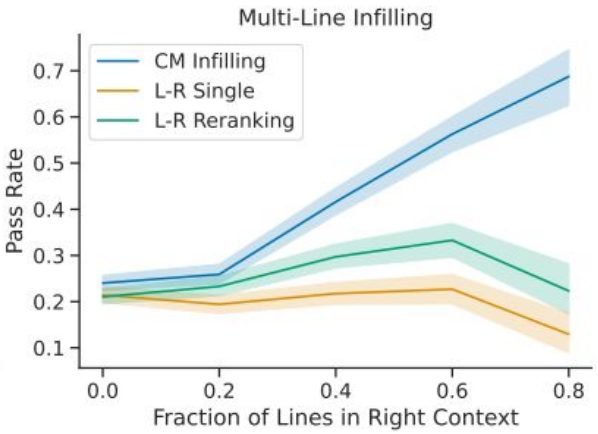
Method	Pass Rate	Exact Match
L-R single	24.9	15.8
L-R reranking	28.2	17.6
CM infilling	38.6	20.6
PLBART	13.1	—
code-cushman-001	30.8	17.4
code-davinci-001	37.8	19.8

(b) Multi-line infilling.

Infilling Results

When the context increases, CM infilling pass rate increases





CodeXGLUE Python Docstring generation BLEU scores

InCoder performs slightly worse compared to other models, but was in a zero-shot setting

Method	BLEU
Ours: L-R single	16.05
Ours: L-R reranking	17.14
Ours: Causal-masked infilling	18.27
RoBERTa (Finetuned)	18.14
CodeBERT (Finetuned)	19.06
PLBART (Finetuned)	19.30
CodeT5 (Finetuned)	20.36

Return Type Function Results

Method	Accuracy
Left-to-right single	12.0
Left-to-right reranking	12.4
Causal-masked infilling	**58.1**

(a) Results on the test set of the benchmark that we construct from CodeXGLUE.

Method	Precision	Recall	F1
Ours: Left-to-right single	30.8	30.8	30.8
Ours: Left-to-right reranking	33.3	33.3	33.3
Ours: Causal-masked infilling	**59.2**	**59.2**	**59.2**
TypeWriter (Supervised)	54.9	43.2	48.3

(b) Results on a subset of the TypeWriter's OSS dataset (Pradel et al., 2020). We include examples from which we were able to obtain source files, successfully extract functions and types, that have non-None return type hints, and that were not included in our model's training data.

Variable Name Prediction

- Results on the variable renaming benchmark from CodeXGLUE
- Benefits from using CM infilling

Method	Accuracy
Left-to-right single	18.4
Left-to-right reranking	23.5
Causal-masked infilling	30.6

Ablation Experiments

- Comparing the 1.3B, 2.3B and 6.7B CM models

#	Size (B)	Obj.	Training Data	Data Size	Train Tokens	Train Compute	HumanEval Pass@1	MBPP Pass@1
1)	6.7	CM	multi lang + SO	204 GB	52 B	3.0 Z	15	19.4
2)	1.3	CM	multi lang + SO	204 GB	52 B	0.6 Z	8	10.9
3)	1.3	LM	multi lang + SO	204 GB	52 B	0.6 Z	6	8.9
4)	1.3	LM	Python + SO	104 GB	25 B	0.3 Z	9	9.8
5)	1.3	LM	Python	49 GB	11 B	0.1 Z	5	6.1
6)	2.3	LM	multi lang + SO	204 GB	52 B	1.1 Z	9	12.7

Limitations

- Massive model - 6.7B, trained on 248 V100 GPUs for 24 days
- Limited dataset
- Evaluation metrics - example: exact matching
- Accuracy still quite low in various tasks

Future Work

- Explore with more parameters, data, and training steps
- Enable a chat-style feature
- Expand with different data sources

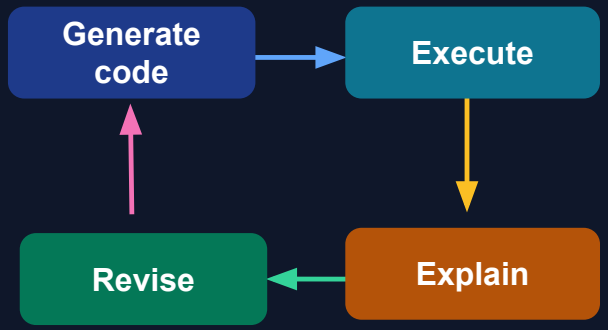
Conclusion

- InCoder shows promise for editing code in between compared to other options
- InCoder can perform tasks in zero-shot settings
- Need for more diverse datasets
- InCoder provides a start for code infilling models

Teaching Large Language Models to Self-Debug

Teach the model to explain, execute, and revise its own code.

Xinyun Chen · Maxwell Lin · Nathanael Schärli · Denny Zhou
Google DeepMind / UC Berkeley · arXiv 2023



Key claim

Iterative self-repair can outperform brute-force sampling, even when unit tests are absent.

Talk roadmap

1. Motivation and method
2. Worked examples
3. Experimental results
4. Critical assessment

Why this paper matters

MOTIVATION

From one-shot generation to iterative self-repair

Observation 1

Large language models are strong code generators, but complex tasks often fail on the first try.

Observation 2

Prior fixes usually rely on either many sampled candidates with reranking, or separate verifier / repair models trained for the task.

Paper thesis

Reuse the failed program. Ask the model to explain what the code does, compare that behavior against the task, and then revise.

Typical brute-force approach

Problem
↓
Sample many programs
↓
Select one by execution or reranking
↓
Discard the rest

→

SELF-DEBUGGING

Problem
↓
Generate one candidate
↓
Execute + explain its behavior
↓
Produce feedback
↓
Revise the same program

Core intuition

Humans rarely write perfect code in one attempt. Debugging is part of programming; the paper treats it as part of inference too.

Chen et al., 2023

SELF-DEBUGGING loop

METHOD

Generation → explanation → feedback → revision

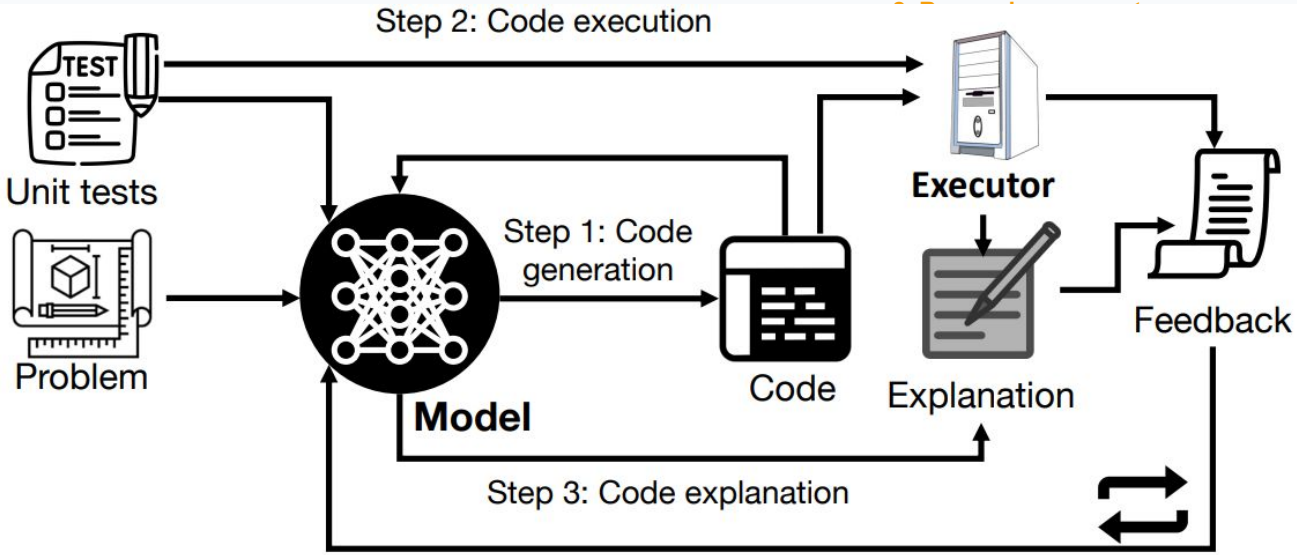


Figure 1: SELF-DEBUGGING for iterative debugging using a large language model. At each debugging step, the model first generates new code, then the code is executed and the model explains the code. The code explanation along with the execution results constitute the feedback message, based on which the model infers the code correctness and then adds this message to the feedback. The feedback message is then sent back to the model to perform more debugging steps. When unit tests are not available, the feedback can be purely based on code explanation.

Chen et al., 2023

Feedback regimes

METHOD

The paper studies four increasingly informative ways to drive revision

Simple feedback
"The code above is wrong. Please fix it."

Minimal supervision: only correctness, no localization.

Unit test feedback (UT)
Failed assertion, actual output, or runtime error.

Grounded signal from execution.

UT + explanation
The model explains the original and translated code line by line before revising.

UT + trace
The model predicts a step-by-step execution trace for the failed call.

more information for diagnosis

Spider (text-to-SQL)
No unit tests are available. The crucial signal is explanation: the model compares the inferred SQL semantics against the question.

TransCoder
Full unit tests are available. Execution identifies failed cases, and explanation or trace helps localize the bug.

MBPP
Only one test is shown in the prompt. Even after passing it, the model is asked whether the code is truly correct.

Subtle but important
Execution traces and line-by-line explanations are generated by the model itself; they do not require privileged access to hidden execution states.

Chen et al., 2023

Spider example: self-debugging SQL without unit tests

METHOD

A worked example of “rubber duck debugging”

Step 1: Question explanation

Infer the return type of the question.

[Question]

Which customers have both "On Road" and "Shipped" as order status? List the customer names.

[Question Explanation]

"List the customer names" returns 1 column. The question returns the customer names who have both "On Road" and "Shipped" as order status. So the question returns 1 column.

Step 2: Code explanation

Summarize the return type of the execution.

[SQL]

SELECT ustomers.customer_name FROM customers JOIN orders ON customers.customer_id = orders.customer_id
WHERE orders.order_status = "On Road" OR orders.order_status = "Shipped"

Execution:

| George |

[SQL Explanation]

The execution of the SQL query above would return a table with 1 column.
The first column, "customers.customer_name" would contain the customer names.
With "customers JOIN orders", the table would contain the data about customers with orders.
With "WHERE orders.order_status = 'On Road' OR orders.order_status = 'Shipped'", the table filters the records to only include customers who have order status "On Road" or "Shipped".
So the SQL query returns a table with 1 column, the customer names who have the order status "On Road" or "Shipped".

Step 3: Self-Debugging with explanation

[Question]

[Explanation summary]

As in your explanation, the SQL query returns a table with 1 column, the customer names who have the order status "On Road" or "Shipped". The question returns 1 column, the customer names who have both "On Road" and "Shipped" as order status.

[Feedback]

So the SQL prediction above is wrong. Please fix the SQL.

[Revised SQL #1]

SELECT customers.customer_name FROM customers JOIN orders ON customers.customer_id = orders.customer_id
WHERE orders.order_status = "On Road" AND orders.order_status = "Shipped"

[Explanation summary #1]

As in your explanation, the SQL query returns an empty table, because the "WHERE" clause contains mutually exclusive conditions. The question returns 1 column, the customer names who have order status of both "On Road" and "Shipped”.

[Feedback #1]

So the SQL prediction above is wrong. Please fix the SQL.

[Revised SQL #2]

SELECT customers.customer_name FROM customers JOIN orders ON customers.customer_id = orders.customer_id WHERE orders.order_status = "On Road" INTERSECT SELECT customers.customer_name FROM customers JOIN orders ON customers.customer_id = orders.customer_id WHERE orders.order_status = "Shipped"

[Explanation summary #2]

As in your explanation, the SQL query returns a table with 1 column, the customer names who have the order status "On Road" and "Shipped". The question returns 1 column, the customer names who have order status of both "On Road" and "Shipped". So the question returns 1 column.

[Feedback #2]

So the SQL prediction above is correct!

Figure 3: An example of SELF-DEBUGGING prompting for text-to-SQL generation. The SQL query, explanation and feedback are all predicted by the model. When the returned table has more than 2 rows, only the first 2 rows are included in the prompt. Database information is omitted in the figure for clarity, and we present the full prompts in Appendix E.

Key insight: the explanation acts like a semantic verifier when tests are absent.

Chen et al., 2023

Example: TransCoder task setup before debugging

EXAMPLE

Before the model can debug, it must first solve the raw TransCoder problem: translate a C++ program into Python while preserving the behavior checked by the unit tests.

C++ Program

```
string caesar_cipher ( string text,
int s ) {
  string result = "";
  for ( int i = 0;
  i < text . length ( );
  i ++ ) {
    if ( isupper ( text [ i ] ) )
    result += char ( int ( text [ i ]
    + s - 65 ) % 26 + 65 );
    else result += char ( int ( text [
     i ] + s - 97 ) % 26 + 97 );
  }
  return result;
}
```

Python Program

```
def caesar_cipher(text, s):
    result = ''
    for i in range(len(text)):
        char = text[i]
        if char.isupper():
            result += chr(((((ord(char
            ) + s) - 65) % 26) + 65))
        else:
            result += chr(((((ord(char
            ) + s) - 97) % 26) + 97))
    return result
```

Unit Tests

```
assert caesar_cipher('35225904', 2) == 'ikhhkofj'
... (8 unit tests omitted)
assert caesar_cipher('11', 93) == 'tt'
```

Figure 4: An example from the TransCoder dataset. The problem description contains the C++ program and unit tests, and the model is required to predict the Python program.

Why this matters: Figure 4 defines the translation task; Figure 5 shows the next step, where failed tests are converted into self-debugging feedback.

With unit tests, execution localizes the bug

METHOD

Code translation example: C++ integer division versus Python float division

Original C++ logic

Integer division is implicit in C++ here

```
for (int i = 5; n / i >= 1; i *= 5)
    count += n / i;
```

Buggy Python translation

Same surface syntax, different semantics

```
i = 5
while (n / i) >= 1:
    cnt = cnt + (n / i)
    i *= 5
```

Unit-test feedback

```
assert count_trailing_zeroes_factorial_number(9) == 1
Actual result: 1.8
```

What execution tells us

The failure is not a random bug: Python “/” returns a float, while the C++ code behaves like integer division for these inputs.

Minimal fix

```
i = 5
while (n / i) >= 1:
    cnt = cnt + (n // i)
    i *= 5
```

MBPP nuance

Only one unit test is shown in the prompt, so the model is still asked to decide whether passing that test means the code is truly correct.

General lesson

Unit tests ground the repair with concrete evidence; explanation or trace then helps the model pinpoint which semantic detail is wrong.

Chen et al., 2023

Experimental setup

EXPERIMENTS

Three tasks, four model families, one repair loop

Spider
Task: text-to-SQL generation
Input: natural-language question + database schema
Available tests: none
Evaluation: prediction accuracy on the development set

TransCoder
Task: C++ → Python translation
Input: C++ function + 10 unit tests per problem
Test set: 560 problems
Evaluation: pass@1

MBPP
Task: text-to-Python generation
Input: problem statement + first unit test
Test set: 500 problems, 3 tests each
Evaluation: remaining hidden tests

Models
code-davinci-002 (Codex), GPT-3.5, GPT-4, and StarCoder 15.5B.

Decoding protocol
Greedy decoding when starting from one program; temperature 0.7 when sampling multiple initial programs.

Selection and turns
If multiple initial programs are sampled, the paper selects by execution-consistent outputs, then debugs greedily for up to 10 turns; most successes occur within 3.

Why this design matters
The same framework is tested in a no-test setting, a full-test setting, and a partial-test setting.

Chen et al., 2023

Main results

EXPERIMENTS

Table 1 summarizes the paper's headline comparisons

Table 1: Comparing SELF-DEBUGGING to prior ranking techniques.

(a) Results on the Spider development set.

	Spider (Dev)
w/ training	
T5-3B + N-best Reranking	80.6
LEVER (Ni et al., 2023)	81.9
Prompting only w/o debugging	
Coder-Reviewer	74.5
MBR-Exec	75.2
SELF-DEBUGGING (this work)	
Codex	81.3
+ Expl.	**84.1**

(b) Results on MBPP dataset.

	n samples
Prior work	
MBR-Exec	63.0 ($n = 25$)
Reviewer	66.9 ($n = 25$)
LEVER	68.9 ($n = 100$)
SELF-DEBUGGING (this work)	
Codex	72.2 ($n = 10$)
Simple	73.6
UT	75.2
UT + Expl.	**75.6**

Spider takeaway
Prompting-only SELF-DEBUGGING reaches 84.1 and beats both prompting rerankers and trained baselines reported in Table 1.

MBPP takeaway
75.6 with only 10 initial samples beats prior methods that needed 25 or even 100 candidates.

What actually drives the gain? Feedback quality.

EXPERIMENTS

Codex results isolate the effect of the feedback signal

Table 2: Results of SELF-DEBUGGING with different feedback formats.

(a) Results on the Spider development set.

Spider	Codex	GPT-3.5	GPT-4	StarCoder
Baseline	81.3	71.1	73.2	64.7
Simple	81.3	**72.2**	73.4	**64.9**
+Expl.	**84.1**	**72.2**	**73.6**	**64.9**

(b) Results on TransCoder.

TransCoder	Codex	GPT-3.5	GPT-4	StarCoder
Baseline	80.4	89.1	77.3	70.0
Simple	89.3	91.6	80.9	72.9
UT	91.6	**92.7**	88.8	76.4
+ Expl.	**92.5**	**92.7**	**90.4**	**76.6**
+ Trace.	87.9	92.3	89.5	73.6

(c) Results on MBPP.

MBPP	Codex	GPT-3.5	GPT-4	StarCoder
Baseline	61.4	67.6	72.8	47.2
Simple	68.2	70.8	78.8	50.6
UT	69.4	72.2	**80.6**	52.2
+ Expl.	69.8	**74.2**	80.4	52.2
+ Trace.	**70.8**	72.8	80.2	**53.2**

Execution matters
With Codex on TransCoder, UT + explanation reaches 92.5 with execution but only 83.9 without unit-test execution (Table 3).

Chen et al., 2023

Sample efficiency on Spider

EXPERIMENTS

Figure 6 on Spider with Codex

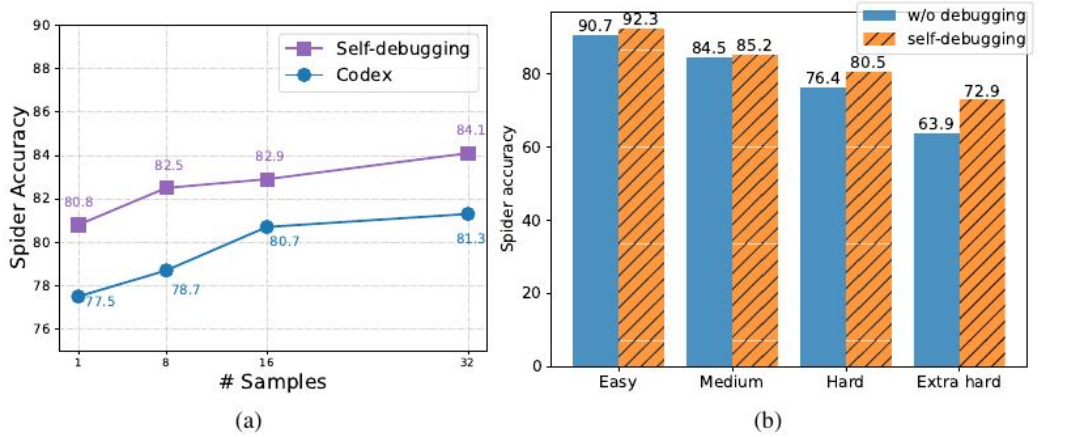


Figure 6: Ablation studies on the Spider development set with Codex. (a) Accuracies with different numbers of initial samples. (b) Breakdown accuracies on problems with different hardness levels.

Sample efficiency
One self-debugged sample roughly matches the baseline that uses 16 initial samples.

Search reduction
Eight self-debugged samples outperform the baseline that uses 32 samples.

Hardest subset
Extra-hard Spider improves from 63.9 to 72.9, a gain of 9.0 points.

Interpretation
Repairing a near-miss program is often cheaper and more effective than sampling many more candidates from scratch.

Turn budget
The paper notes that most successful debugging finishes within 1–3 turns.

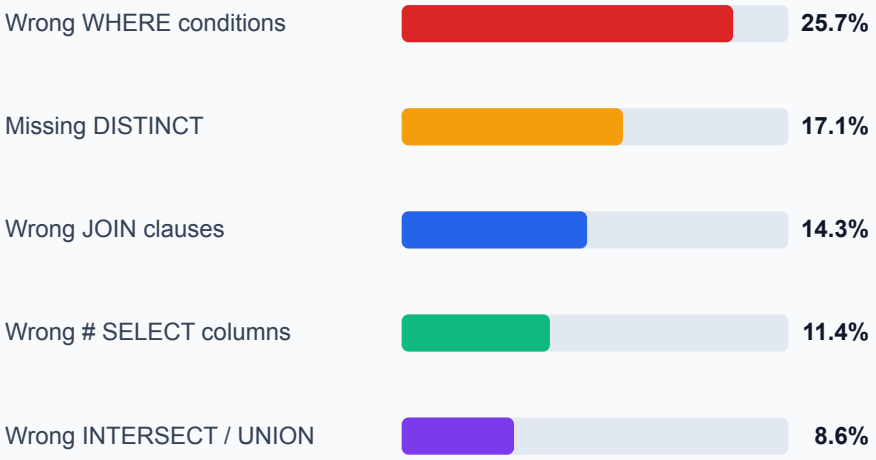
Chen et al., 2023

Error analysis: the method repairs near-miss semantics

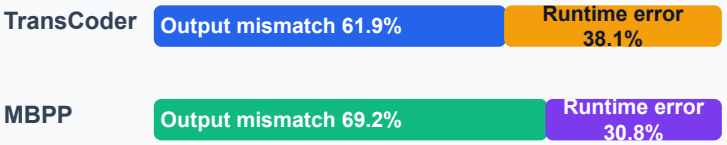
ANALYSIS

What kinds of bugs does self-debugging actually fix?

Top SQL error types fixed on Spider



Successful fixes on code tasks



Example of a subtle Spider repair

Fixing an extra-hard query by adding the missing official-language condition

```
... WHERE language = "English"
AND isofficial = "T"
INTERSECT
... WHERE language = "French"
AND isofficial = "T"
```

Mechanistic reading

The loop works best when the initial program is close to correct: missing conditions, wrong operators, wrong column counts, or language-specific semantic mismatches.

Not magic synthesis

Most gains come from repairing local semantic errors rather than inventing an entirely new solution from scratch.

Chen et al., 2023

Assessment and open questions

ANALYSIS

What is genuinely strong here, and what still remains unresolved?

What is strong

- Conceptual contribution: it turns explanation into a practical verifier for code.
- Training-free generality: the same prompting idea transfers across SQL, translation, and Python generation.
- Sample efficiency: iterative repair can dominate brute-force sampling with far fewer initial candidates.
- Strong no-test result: the Spider gains show that self-generated semantic explanations can sometimes substitute for external feedback.

What remains open

- Prompt sensitivity is real: in this setup, GPT-4 underperforms Codex on Spider, likely because the prompts are long and few-shot heavy.
- Repair has a test-time cost: execution, explanation, and multiple turns all consume tokens and latency.
- The method seems strongest for near-miss programs; the paper does not show long-horizon debugging on very large codebases.
- Explanation quality is hypothesized to matter, but it is not directly measured as a causal variable.

My research takeaway

The paper argues that coding models should not only generate programs; they should also learn to critique, localize, and repair them during inference.

Chen et al., 2023

Conclusions

Teach models to debug, not just to draft.

1 Reuse failed code
A wrong program is not just an error signal; it is material for diagnosis and revision.

2 Ground + interpret
Execution provides grounding, while explanation exposes the program's semantics.

3 Repair beats search
Iterative self-debugging can outperform sampling many more candidates from scratch.

Thank you

Q&A

Chen et al., 2023

SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering

John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, Ofir Press

Presented by: Grayson Xie

LM Agents for Code Generation

LMs act as agents by iteratively:

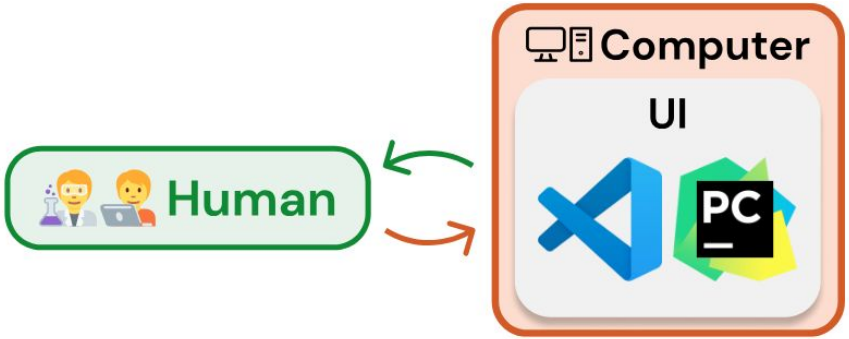
1. Taking actions in an environment
2. Receiving feedback / observations
3. Reasoning about next steps

Applying agents to more complex code tasks like software engineering remains unexplored.

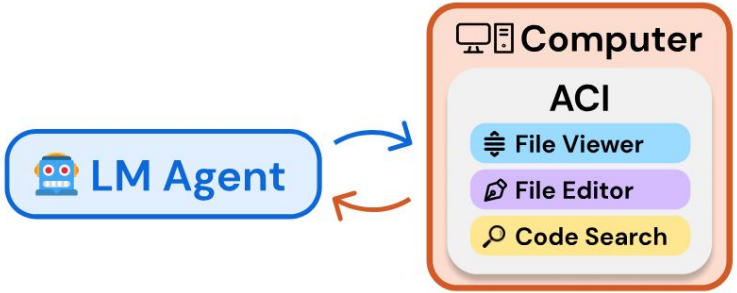
The Problem: No suitable interface for LM

- Human developers use applications like VSCode with powerful tools and extensions
- LM Agents use a raw Linux shell

LMs lack visual perception, have fixed per-token cost, and cannot selectively ignore irrelevant information



Agent-Computer Interface (ACI)



- An abstraction layer between the LM agent and the computer
- Specifies the commands available to the agent
- Controls how environment state is communicated back to the LM
- Manages history to avoid unnecessary context from prior observations

- **Principle 1:** Actions should be simple and easy to understand
- **Principle 2:** Actions should be compact and efficient
- **Principle 3:** Feedback should be informative but concise
- **Principle 4:** Guardrails should mitigate error propagation

SWE-agent's ACI Components

Search & Navigation: Three commands (`find_file`, `search_file`, `search_dir`) that output summarized results.

File Viewer — An interactive windowed viewer (`open`, `goto`, `scroll_up`, `scroll_down`) showing 100 lines at a time with line numbers, file path, and counts of omitted lines above/below.

File Editor — A single `edit n:m <replacement> end_of_edit` command that replaces lines n through m.

Context Management — Observations older than the last 5 turns are collapsed to a single line (preserving action history but removing verbose output). A "no output" message is shown explicitly when a command produces nothing.

SWE-agent's ACI Components

Observation (File Viewer)

```
[File: solvers/diophantine.py (3093 lines total)]
(400 lines above)
401:    elif total_degree == 3:
402:        diop_type = "cubic_thue"
403:
404:    elif (total_degree > 3):
405:        if all(coeff[k] == 1):
406:            diop_type = 'general_sum_even'
407:
408:    if diop_type is not None:
409:        return var, coeff, diop_type
(2684 lines below)
```

(a) Observation from the file viewer.

Response (Thought + Edit Action)

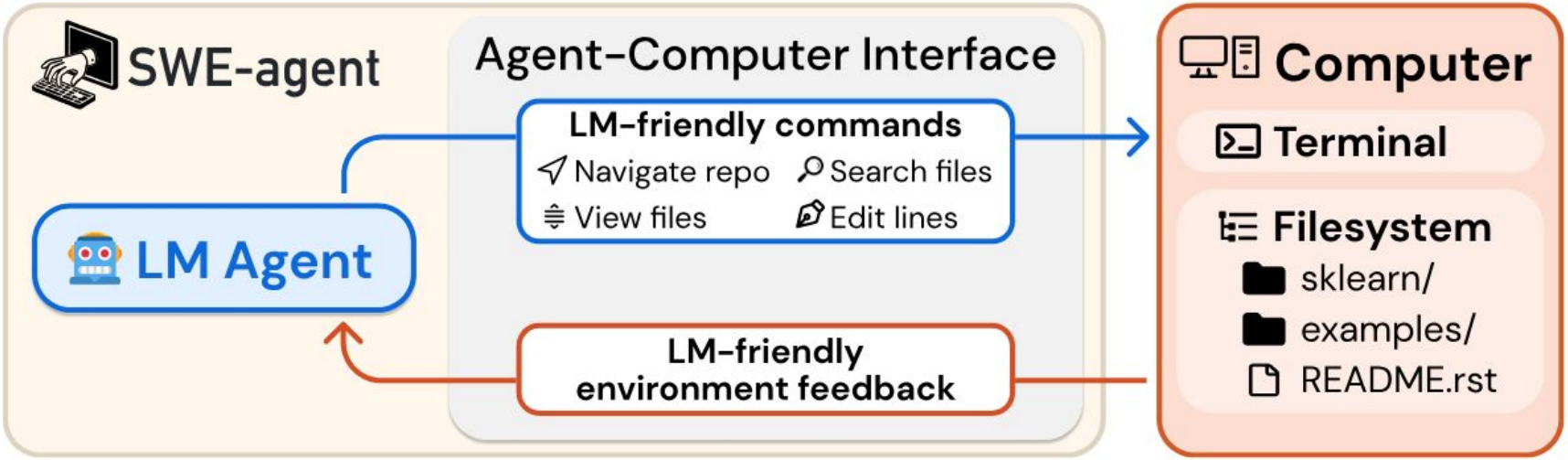
We need to add a check to ensure that all powers are the same. Let's edit the code to include this check.

```
edit 404:407
    elif (total_degree > 3 and
          len(set(k.exp[k.is_Pow])) == 1):
        if all(coeff[k] == 1):
            diop_type = 'general_sum_even'
end_of_edit
```

(b) Action using the edit interface.

Figure 3: The file viewer and edit command are integrated. (a) The file viewer shows the agent the open file's content with line numbers. (b) The agent invokes the edit function to replace lines 404-407 in the open file. After the edit, the file viewer shows the agent the now updated version of the file.

Overview


SWE-agent
Agent-Computer Interface
LM-friendly commands
Navigate repo
Search files
View files
Edit lines
LM Agent
LM-friendly environment feedback
Computer
Terminal
Filesystem
sklearn/
examples/
README.rst

Evaluation

Benchmarks:

- **SWE-bench** (2,294 real GitHub issues from 12 Python repos)
- **SWE-bench Lite** (300-instance canonical subset), for ablation
- **HumanEvalFix** (a short-form code debugging benchmark)

Models:

- **GPT-4 Turbo** (128k context), **Claude 3 Opus** (200k context)
- Open models like Llama 3 and DeepSeek Coder performed poorly, partly due to small context windows.

Baselines:

- Non-interactive, RAG baselines
- Shell-only. Asks the LM to resolve the issue by interacting with a shell process on Linux

Main Results

Model	SWE-bench		SWE-bench Lite	
	% Resolved	$ Avg. Cost	% Resolved	$ Avg. Cost
RAG				
w/ GPT-4 Turbo	1.31	0.13	2.67	0.13
w/ Claude 3 Opus	3.79	0.25	4.33	0.25
Shell-only agent				
w/ GPT-4 Turbo	-	-	11.00	1.46
w/o Demonstration	-	-	7.33	0.79
SWE-agent				
w/ GPT-4 Turbo	**12.47**	1.59	**18.00**	1.67
w/ Claude 3 Opus	10.46	2.59	13.00	2.18

HumanEvalFix Results

- Pass@1 results with GPT-4 Turbo: Python: 87.7% | JavaScript: 89.7% | Java: 87.9%
- Compared to prior state-of-the-art:
  - GPT-4 (zero-shot): 47.0% / 48.2% / 50.0%
  - WaveCoder-DS-6.7B: 57.9% / 52.4% / 57.3%
  - SWE-agent nearly doubles best prior results

Model	Python	JS	Java
CodeLLaMa-instruct-13B	29.2	19.5	32.3
GPT-4	47.0	48.2	50.0
DeepseekCoder-CodeAlpaca-6.7B	49.4	51.8	45.1
WaveCoder-DS-6.7B	57.9	52.4	57.3
SWE-agent w/ GPT-4 Turbo	**87.7**	**89.7**	**87.9**

Ablation

Editor		Search		File Viewer		Context	
`edit` action	15.0 ↓ 3.0	Summarized 🤠	18.0	30 lines	14.3 ↓ 3.7	Last 5 Obs. 🤠	18.0
w/ linting 🤠	18.0	Iterative	12.0 ↓ 6.0	100 lines 🤠	18.0	Full history	15.0 ↓ 3.0
No `edit`	10.3 ↓ 7.7	No search	15.7 ↓ 2.3	Full file	12.7 ↓ 5.3	w/o demo.	16.3 ↓ 1.7

Every ACI component contributes meaningfully; the full system is greater than the sum of its parts

HumanEvalFix Results

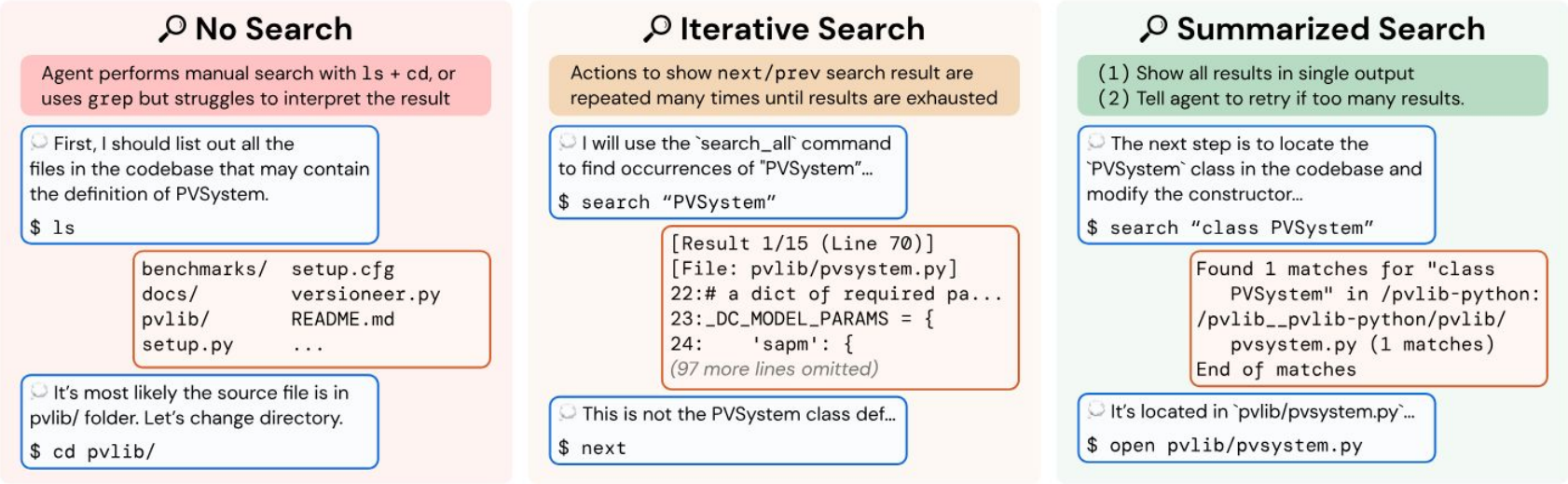


Figure 5: Three different Search interfaces for task instance `pvlib__pvlib-python-1224`. In Shell-only, an agent performs localization using only standard bash commands and utilities. Compared to *Iterative* search, *Summarized* search shows an exhaustive list of search results and provides guidance on refining under-specified queries.

HumanEvalFix Results

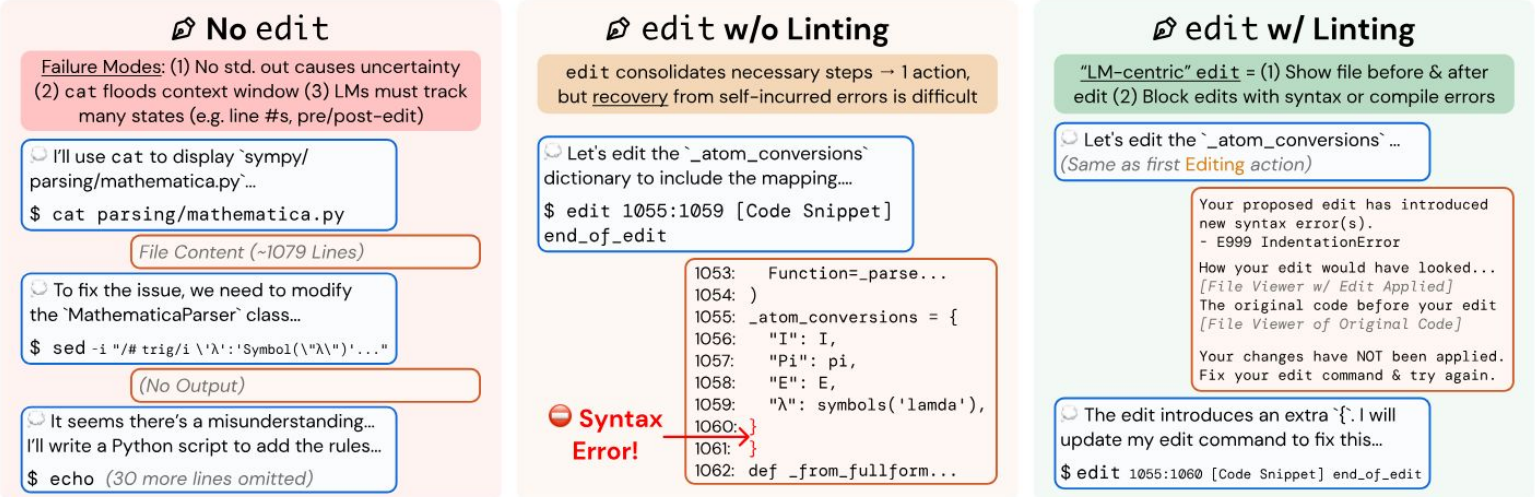


Figure 6: Three different Edit interfaces for task instance `sympy__sympy-24102`. Editing with bash commands requires several actions to successfully modify a file. The *Editing* component defines an `edit` command that leverages the File Viewer component to replace the bash style of editing workflow with a single command. *Linting* is beneficial for stymieing cascading errors that often start with an error-introducing edit by the agent.

Agent Behavior Patterns

- Consistent problem-solving pattern emerges:
  - Step 1: Reproduce the bug
  - Step 2: Localize the issue
  - Step 3: Edit-then-execute loops
- Editing remains challenging:
  - 51.7% of trajectories have at least one failed edit
  - Recovery probability: 90.5% on first try, drops to 57.2% after one failure
- Agents succeed fast and fail slow:
  - Successful runs: median $1.21 cost, 12 steps
  - Failed runs: median $2.52 cost, 21 steps

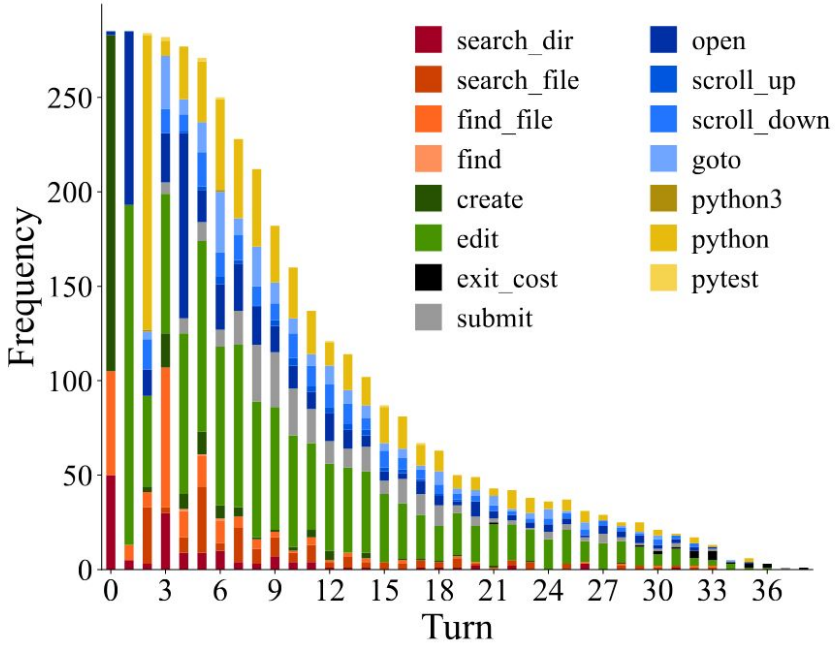
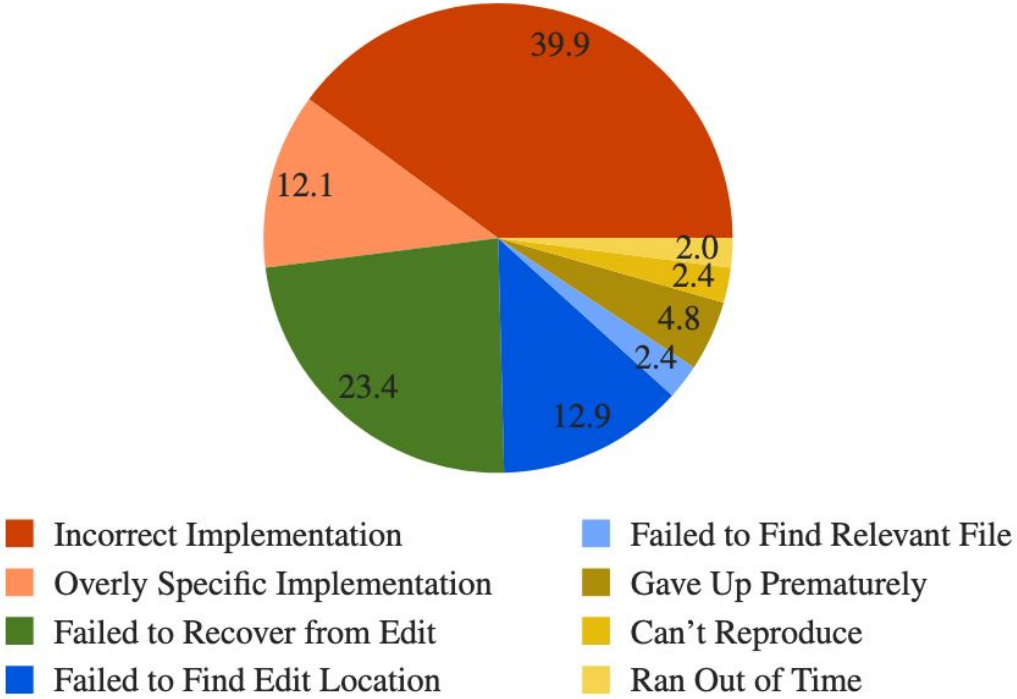


Figure 7: The frequency with which actions are invoked at each turn by SWE-agent w/ GPT-4 for task instances that it solved on the SWE-bench full test set (286 trajectories).

Failure Analysis

- 248 unresolved Lite instances by GPT-4o
  - Incorrect implementation: 39.9%
  - Failed to recover from edit errors: 23.4%
  - Over half of failures are capability limitations, not interface issues -> improving the underlying model is the next step



Limitations

- **More integration could be applied**: Adding more tools such as web browsing or static analysis could provide more information.

- **ACI design is manual:** Unlike prompts which can already be automatically generated, the ACI was hand-designed through qualitative inspection rather than learned or automatically optimized.

- **Only focused on programmatic tasks:** SWE-agent only handles tasks like software engineering and code generation. The effectiveness of ACI design on other areas are not tested.

Conclusion

- Introduces the Agent-Computer Interface (ACI)
- Proposes Four ACI principles: simple actions, compact operations, concise feedback, and guardrails
- Designs SWE-agent, which outperforms existing RAG and Shell-only baselines.
- Draws a parallel between HCI and ACI research: studying how agents use interfaces can reveal insights about LM capabilities and limitations

Overall Conclusions

- InCoder is an LLM that uses both left and right context to code
- Teaching LLM to self debug: Self generated explanation + reliable external verifier can improve accuracy and efficiency.
- SWE-agent's ACI yields a 64% improvement over a shell-only baseline with the same underlying LM, showing that how agents interact with tools matters as much as raw model power
- Across all three papers, the primary failure mode is the model producing incorrect solutions, and improving the underlying LM remains the most impactful next step